

Event ticket demand and pricing

A fictional 16-match event, used to show how I segment demand, estimate price response and test a forecast.

SYNTHETIC DATA. ILLUSTRATIVE. NOT CLIENT WORK. Every number on this page comes from a script that invents the data. It does not describe any real event, client, price or method.

TYPE Synthetic case study	TOOLS Python, pandas, NumPy, Matplotlib	DATA Generated, seeded (42)
BUILT Oct 2026		

01

The question

An organiser sells tickets to a 16-match event. How do demand and revenue respond to price for different kinds of buyer, and how well can a simple model forecast matches it has not seen?

02

The data

There is no real data here. `generate_data.py` invents 432 rows (16 matches × 3 seat categories × 3 buyer segments × 3 purchase windows) with a fixed random seed, so anyone can regenerate the same file. It sold 141,845 tickets in total.

- **Match tier** A, B or C, a stand-in for how much people want to go.
- **Buyer segment:** Local, Domestic travel, International.
- **Purchase window:** Early (over 60 days out), Mid (15 to 60), Late (under 15).
- **Capacity:** seats are finite, and 8 match and category combinations sold out.

Because I wrote the generator, I know the true price elasticity of each segment (Local -1.6 , Domestic -1.1 , International -0.6). The model never sees those numbers. That lets me check whether the method recovers them, which real data would never allow.

03

The method

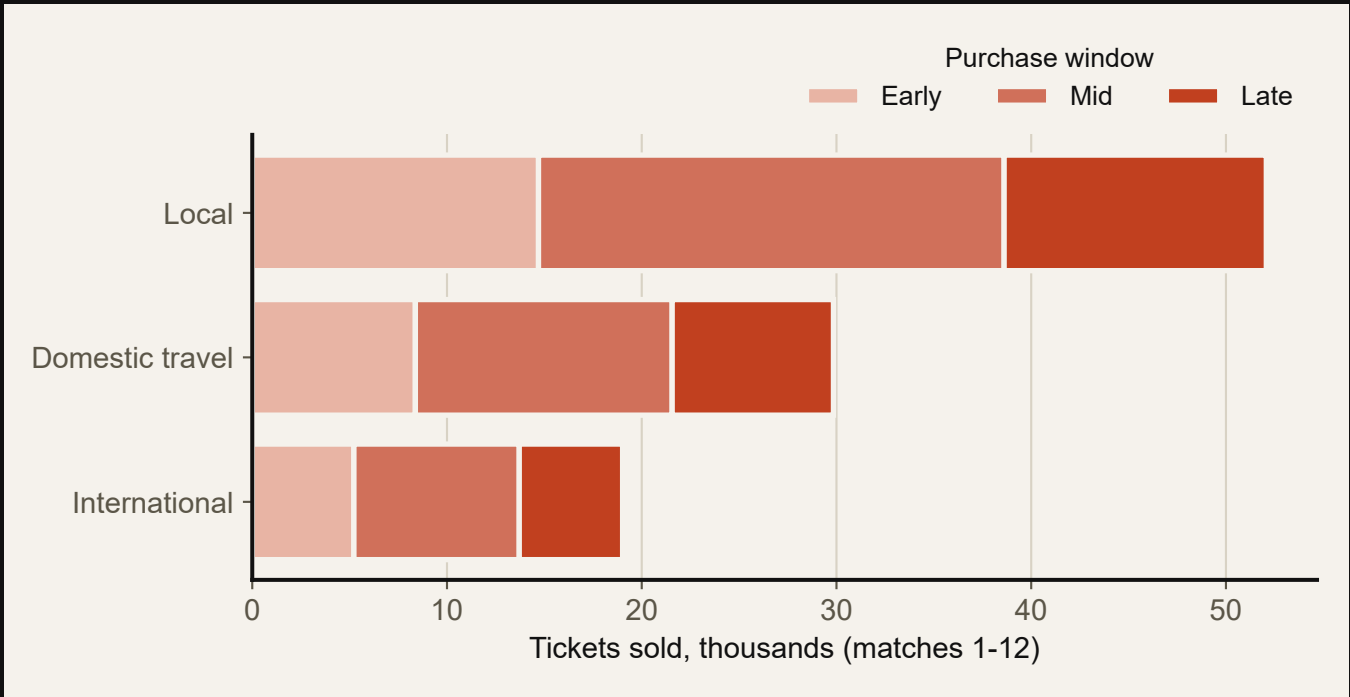
1. **Segment demand** by tier, buyer segment and purchase window, using matches 1 to 12.
2. **Estimate price response** with one log-log regression per segment: log of tickets sold against log of price, controlling for tier, seat category and window. The price coefficient is the elasticity: a 1% price rise changes tickets sold by that many percent. Sold-out groups are left out of the fit because their sales are capped by seats, not by demand.
3. **Forecast the held-out matches** (13 to 16) using the prices actually charged, then cap each match and category at its seats.
4. **Scenarios:** Low and High scale demand by -15% and $+15\%$. That range is my assumption, not something estimated.
5. **Error:** WAPE (weighted absolute percentage error), the sum of absolute errors divided by the sum of actuals. I compare it with a naive forecast that uses the training average for each segment, window and category.
6. **Sensitivity:** change prices on the held-out matches and see what the model says happens to revenue.

04

What I found

Where demand comes from

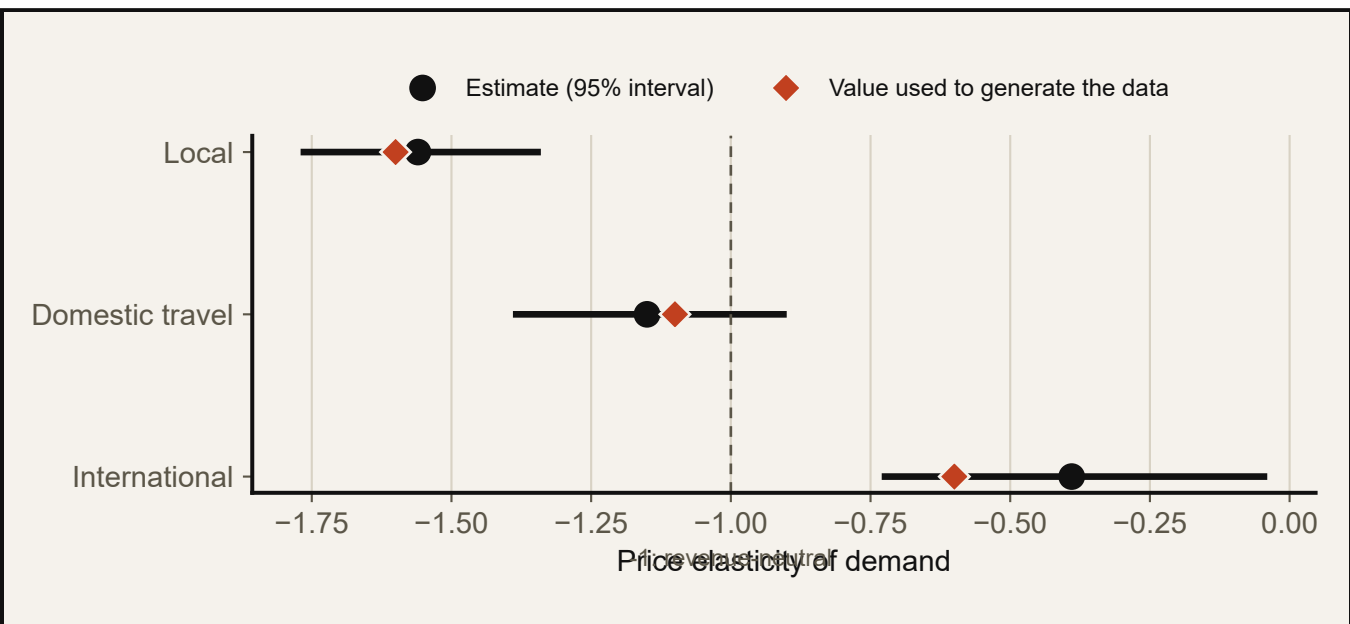
Local buyers account for 52% of tickets in the training matches, domestic travellers 30% and international buyers 19%. In every tier and segment the middle window sells the most.



Tickets sold by segment and purchase window, matches 1 to 12 (synthetic).

Price response

The method recovers the elasticities I built in, within its uncertainty. The international estimate is the least precise: its interval is wide, but it still contains the true value.

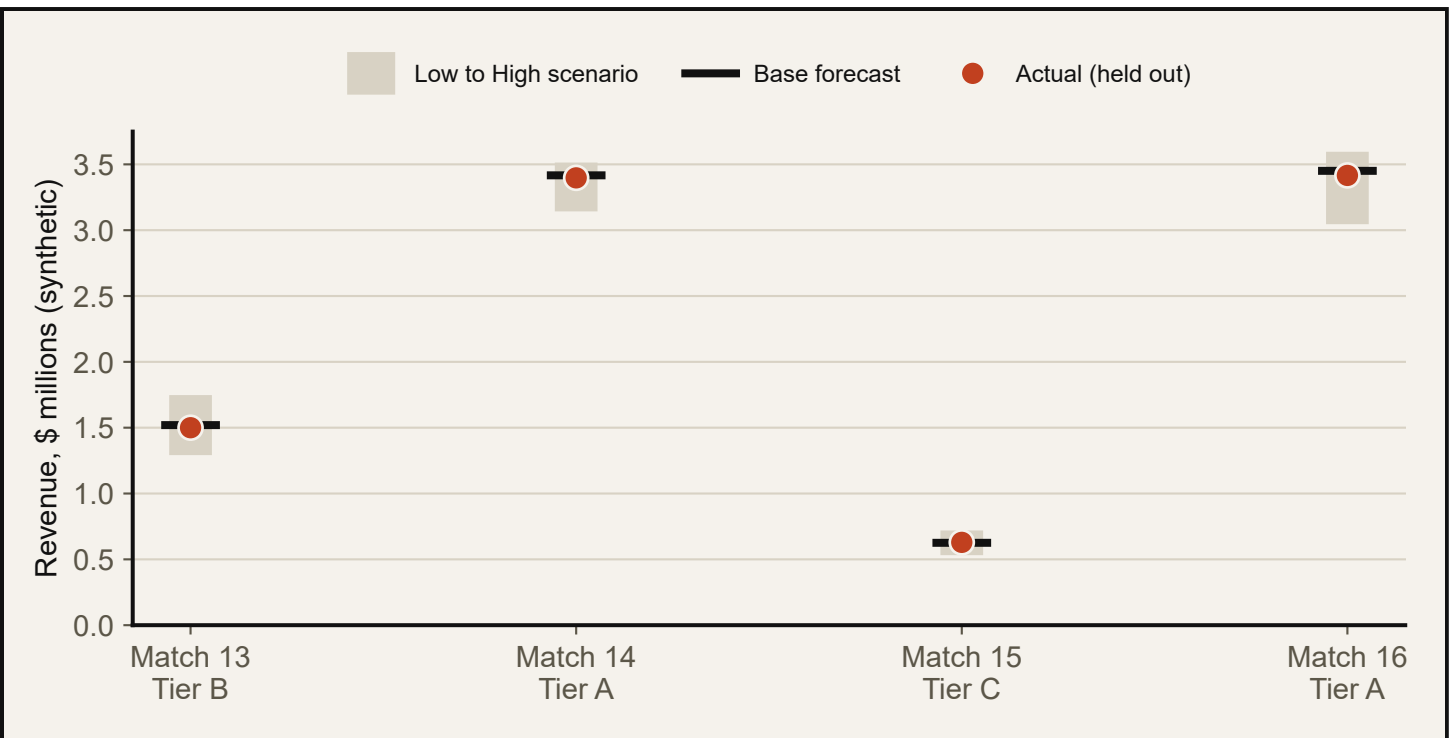


Estimated versus true elasticity by segment (synthetic).

Elasticity by segment (synthetic)			
Segment	Estimate	95% interval	Used to generate data
Local	-1.56	-1.77 to -1.34	-1.6
Domestic travel	-1.15	-1.39 to -0.90	-1.1
International	-0.39	-0.73 to -0.04	-0.6

Forecast on matches the model never saw

Cell-level WAPE on matches 13 to 16 is **8.4%**, against 34.4% for the naive average. Revenue per match is off by 0.9% in aggregate, and total tickets are +0.5% versus actual. Actual revenue fell inside the Low to High band in 4 of 4 matches.



Held-out matches: scenario band, base forecast and actual revenue (synthetic).

Revenue forecast by held-out match (synthetic)					
Match	Low	Base	High	Actual	Inside band
Match 13 (tier B)	\$1.29M	\$1.52M	\$1.75M	\$1.50M	yes
Match 14 (tier A)	\$3.14M	\$3.42M	\$3.51M	\$3.40M	yes
Match 15 (tier C)	\$0.53M	\$0.63M	\$0.72M	\$0.63M	yes
Match 16 (tier A)	\$3.05M	\$3.45M	\$3.60M	\$3.42M	yes

Read these accuracy numbers with suspicion. The model has the same shape as the process that generated the data, so it should do well. On real data I would expect a far larger error.

What a price change would do

Revenue change on the held-out matches (synthetic)			
Price change	Elasticity ×0.7	As estimated	Elasticity ×1.3

Price change	Elasticity $\times 0.7$	As estimated	Elasticity $\times 1.3$
-10%	-6.3%	-4.8%	-3.4%
-5%	-2.9%	-2.2%	-1.4%
+0%	+0.0%	+0.0%	+0.0%
+5%	+2.6%	+1.4%	+0.3%
+10%	+4.8%	+2.0%	-0.8%

With the estimated elasticities, a 5% price rise lifts revenue by +1.4% and a 10% rise by +2.0%. If demand were 30% more price-sensitive than estimated, the 10% rise would lose money. The answer depends on a number I can only estimate, which is why the interval matters.

Revenue change from a 5% price rise, by segment (synthetic)	
Segment	Revenue change
Local	-0.3%
Domestic travel	+1.7%
International	+5.6%

The gain comes from the low-elasticity segments. Raising prices for local buyers is roughly revenue-neutral; for international buyers it is worth about +5.6%.

05

Limitations

- The data is invented, and the model matches the generator. Real forecast error would be larger.
- Prices here vary at random. In real sales, organisers raise prices when demand is high, which biases a naive elasticity estimate toward zero. Separating the two takes price experiments or careful natural variation.
- Elasticity is constant, so large price moves extrapolate badly. There are no competitors, substitution between categories or matches, or drift over time.
- Only four matches are held out, so one accuracy number is thin evidence. The $\pm 15\%$ scenario range is an assumption.

06

What I'd do differently

- Fit a count model (Poisson or negative binomial) instead of least squares on logs, and shrink thin segments toward the overall pattern.
- Backtest over several rolling windows so scenario ranges come from observed error, not an assumed $\pm 15\%$.
- Use real price experiments to estimate elasticity, and optimise prices against seat capacity rather than only testing a flat change.

07

Code and data

- [Code, README and generated data on GitHub](https://github.com/samriddhosaha/samriddha-portfolio/tree/HEAD/case-studies/event-pricing) [↗] [_\(https://github.com/samriddhosaha/samriddha-portfolio/tree/HEAD/case-studies/event-pricing\)](https://github.com/samriddhosaha/samriddha-portfolio/tree/HEAD/case-studies/event-pricing)
- Run it yourself: `python generate_data.py`, then `python analyze.py`. It needs numpy, pandas and matplotlib.

Last updated 6 Oct 2026.

SAMRIDDHA SAHA

Senior Business Analyst

Contact [LinkedIn\(https://www.linkedin.com/in/samriddha-saha-12375a1a0/\)](https://www.linkedin.com/in/samriddha-saha-12375a1a0/) [GitHub\(https://github.com/samriddhosaha/\)](https://github.com/samriddhosaha/)

© 2026 Samriddha Saha